

Lecture 14: Consensus with Raft

CS 739: Distributed Systems

Prof. Andrea C. Arpaci-Dusseau
University of Wisconsin — Madison

Spring'26 Announcements

3/11 + 3/3: Two In-class Midterm Exams

- Reading Comprehension: Haystack – Mean 15.5 / 16
- Multiple Choice
 - Mean: 87 / 114; Max: 106 / 114
 - Quintiles:
 - 95+
 - 90-94
 - 85-89
 - 80-84
 - 63-79

Project 3 – Due 1 week AFTER Spring Break

- Add consensus to KV-Store
- Form group by Sunday, March 22

Wednesday: Two papers: Chubby (Focus) and Zookeeper (Comparison)

Friday: E-Paxos - Performance Eval

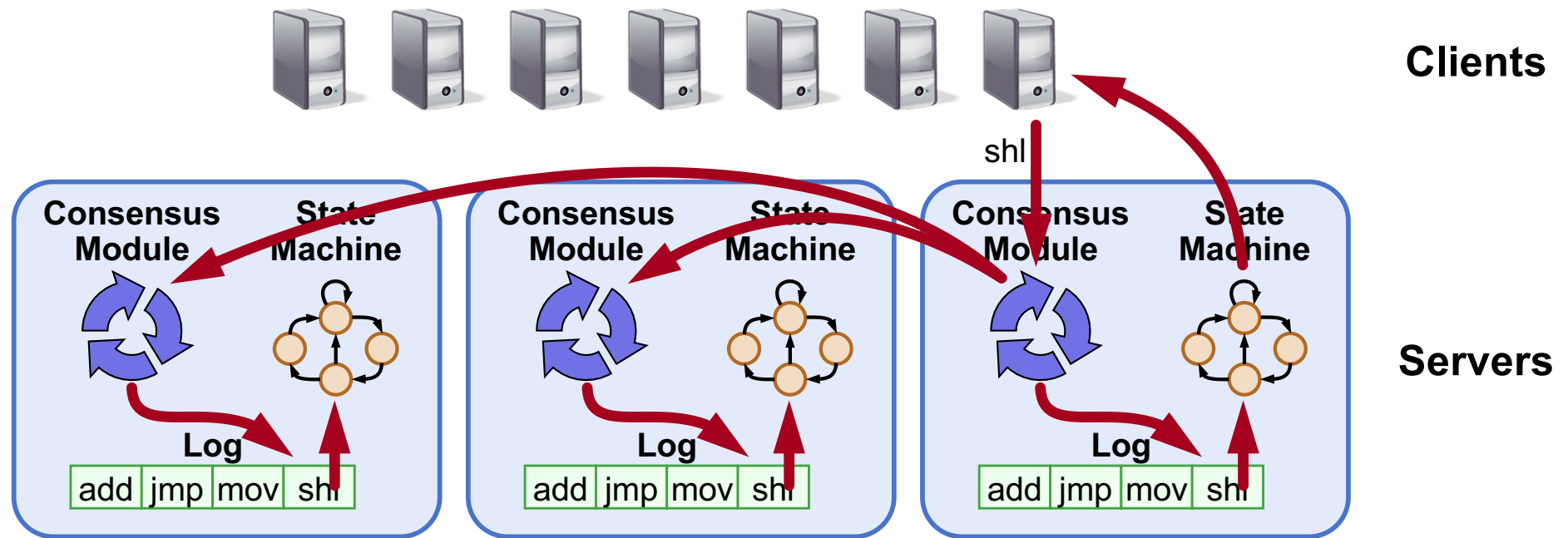
Spring Break

After: Consistency Models

Raft: A Consensus Algorithm for Replicated Logs

- **Diego Ongaro and John Ousterhout**
- **Stanford University**
- **USENIX ATC'14**
- **Contribution of Raft to research community?**
- **Same format of slides as Paxos**
- **Part of a user study (videos available!)**

Goal: Replicated Log



- **Replicated log => replicated state machine**
 - All servers execute same commands in same order
- **Consensus module ensures proper log replication**
- **System makes progress as long as any majority of servers are up**
- **Failure model: fail-stop (not Byzantine), delayed/lost messages**

Approaches to Consensus

Two general approaches to consensus:

- **Symmetric, leader-less:**
 - All servers have equal roles
 - Clients can contact any server
- **Asymmetric, leader-based:**
 - At any given time, one server is in charge, others accept its decisions
 - Clients communicate with the leader
- **Raft uses a leader:**
 - Decomposes the problem (normal operation, leader changes)
 - Simplifies normal operation (no conflicts)
 - More efficient than leader-less approaches

Raft Overview

1. Leader election:

- Select one of the servers to act as leader
- Detect crashes, choose new leader

2. Normal operation (basic log replication)

3. Safety and consistency after leader changes

4. Neutralizing old leaders

5. Client interactions

- Implementing linearizeable semantics

6. Configuration changes:

- Adding and removing servers

Appealing Properties

- Choose most understandable alternative

- Simplify state space

- 1) Don't allow nodes in log
- 2) When it doesn't matter, choose random

• Term, index $\rightarrow$ invariant

• 3 easy states for nodes

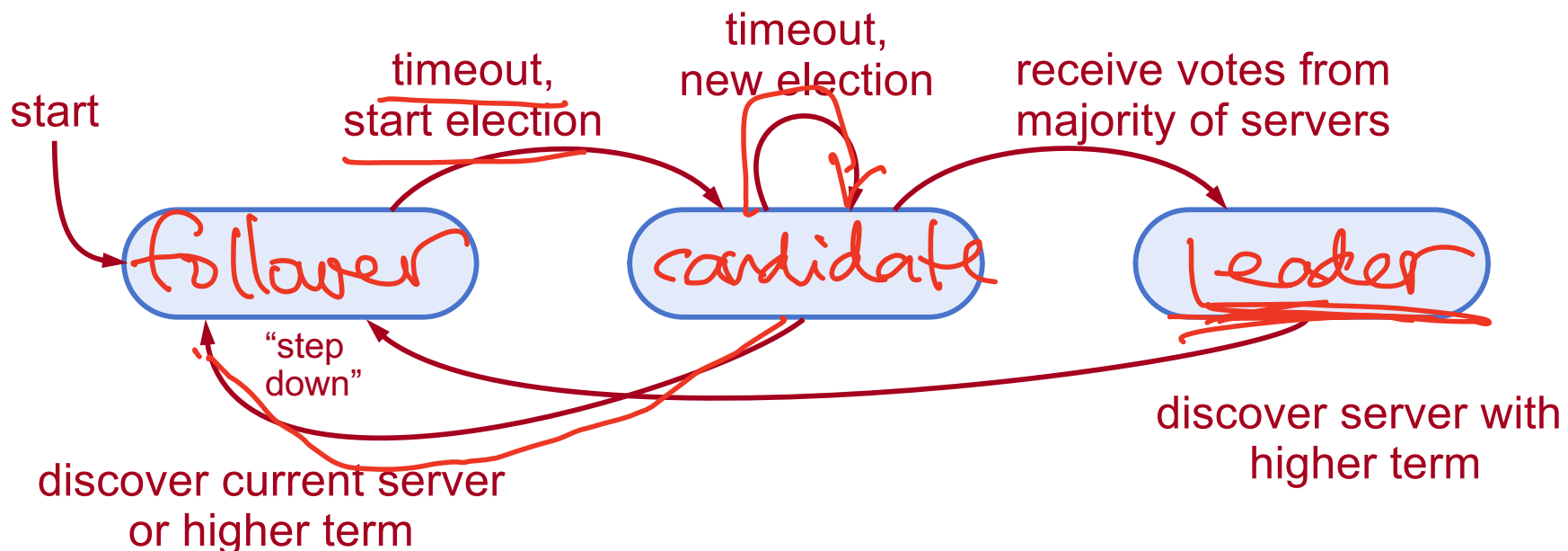
Server States

- At any given time, each server is in one of 3 states:

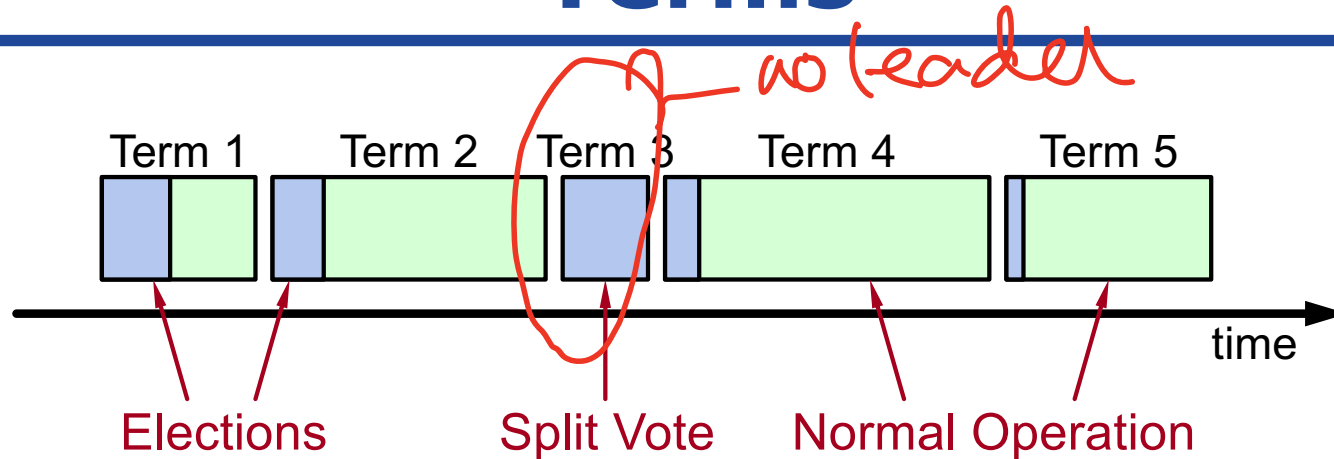
- Leader: handles all client interactions, log replication
- Follower: completely passive (issues no RPCs, responds to incoming RPCs)
- Candidate: used to elect a new leader

N nodes

- Normal operation: *1* leaders, *N-1* followers, *0* candidates



Terms



- **Time divided into terms:**
 - Election
 - Normal operation under a single leader
- **At most 1 leader per term**
- **Some terms have no leader (failed election)**
- **Each server maintains **current term** value**
- **Key role of terms: identify obsolete information**
if RPC from older term, discard!
v leader of

Raft Protocol Summary

Followers

- Respond to RPCs from candidates and leaders.
- Convert to candidate if election timeout elapses without either:
 - Receiving valid AppendEntries RPC, or
 - Granting vote to candidate

Candidates

- Increment currentTerm, vote for self
- Reset election timeout
- Send RequestVote RPCs to all other servers, wait for either:
 - Votes received from majority of servers: become leader
 - AppendEntries RPC received from new leader: step down
- Election timeout elapses without election resolution: increment term, start new election
- Discover higher term: step down

Leaders

- Initialize nextIndex for each to last log index + 1
- Send initial empty AppendEntries RPCs (heartbeat) to each follower; repeat during idle periods to prevent election timeouts
- Accept commands from clients, append new entries to local log
- Whenever last log index $\geq$ nextIndex for a follower, send AppendEntries RPC with log entries starting at nextIndex, update nextIndex if successful
- If AppendEntries fails because of log inconsistency, decrement nextIndex and retry
- Mark log entries committed if stored on a majority of servers and at least one entry from current term is stored on a majority of servers
- Step down if currentTerm changes

Persistent State

Each server persists the following to stable storage synchronously before responding to RPCs:

<u>currentTerm</u>	latest term server has seen (initialized to 0 on first boot)
<u>votedFor</u>	candidateId that received vote in current term (or null if none)
<u>log[]</u>	log entries

Log Entry

term	term when entry was received by leader
index	position of entry in the log
command	command for state machine

RequestVote RPC

Invoked by candidates to gather votes.

Arguments:

candidateId	candidate requesting vote
term	candidate's term
lastLogIndex	index of candidate's last log entry
lastLogTerm	term of candidate's last log entry

Results:

term	currentTerm, for candidate to update itself
voteGranted	true means candidate received vote

Implementation:

- If term > currentTerm, currentTerm $\leftarrow$ term (step down if leader or candidate)
- If term == currentTerm, votedFor is null or candidateId, and candidate's log is at least as complete as local log, grant vote and reset election timeout

AppendEntries RPC

Invoked by leader to replicate log entries and discover inconsistencies; also used as heartbeat.

Arguments:

term	leader's term
leaderId	so follower can redirect clients
prevLogIndex	index of log entry immediately preceding new ones
prevLogTerm	term of prevLogIndex entry
entries[]	log entries to store (empty for heartbeat)
commitIndex	last entry known to be committed

Results:

term	currentTerm, for leader to update itself
success	true if follower contained entry matching prevLogIndex and prevLogTerm

Implementation:

- Return if term < currentTerm
- If term > currentTerm, currentTerm $\leftarrow$ term
- If candidate or leader, step down
- Reset election timeout
- Return failure if log doesn't contain an entry at prevLogIndex whose term matches prevLogTerm
- If existing entries conflict with new entries, delete all existing entries starting with first conflicting entry
- Append any new entries not already in the log
- Advance state machine with newly committed entries

i don't put!

heartbeats at - no ops

common case

if informs followers of new log entries (committed)

2) discover inconsistencies & fix

3 roles

Heartbeats and Timeouts

Who needs to know who is alive?

followers need to know leader alive

Servers start up as *followers*

Followers expect to receive RPCs from *leader*
or *Candidates*

Leader must send **heartbeats** to maintain authority

- empty *AppendEntry* RPCs

If **electionTimeout** elapses with no RPCs:

- Follower assumes leader has crashed
- Follower starts new election
- Timeouts typically 100-500ms

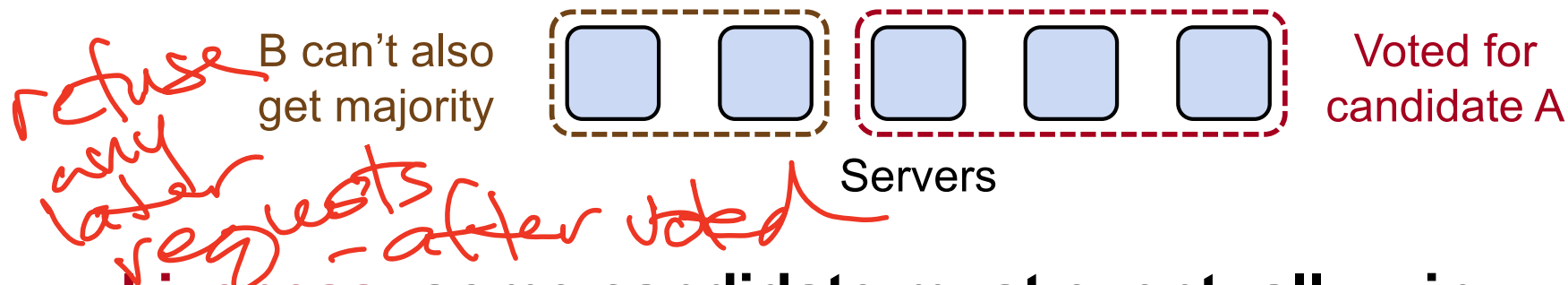
Election Basics

- Increment current term - *persistent*
- Change to *candidate* state
- Vote for self
- Send RequestVote RPCs to all other servers, retry until either:
 1. Receive votes from majority of servers: *- later - who votes to you!*
 - Become *leader*
 - Send AppendEntries heartbeats to all other servers
 2. Receive RPC from valid leader,
 - Return to *follower* state
 3. No one wins election (election timeout elapses):
 - Increment *current term*, start new election

Elections, cont'd

- **Safety:** allow at most one winner per term

- Each server gives out only one vote per term
- Ensure can handle crashes by keep vote persistent
- Two different candidates can't accumulate majorities in same term



- **Liveness:** some candidate must eventually win

- Choose election timeouts randomly in $[T, 2T]$
- One server usually times out and wins election before others wake up
- Works well if $T \gg$ broadcast time

*who gets to be leader
→ later!*

Raft Overview

1. Leader election:

- Select one of the servers to act as leader
- Detect crashes, choose new leader

2. Normal operation (basic log replication)

3. Safety and consistency after leader changes

4. Neutralizing old leaders

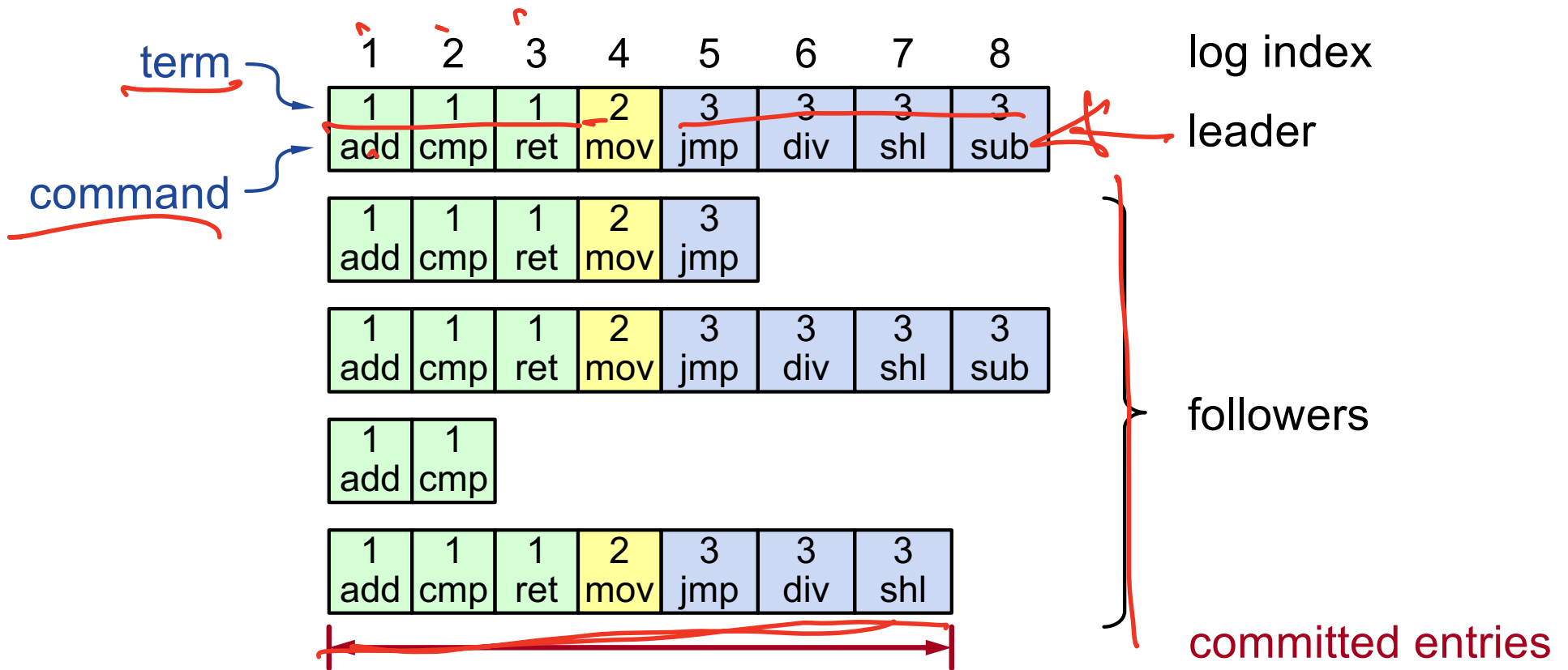
5. Client interactions

- Implementing linearizeable semantics

6. Configuration changes:

- Adding and removing servers

Log Structure



- Log entry = index, term, command
- Log stored on stable storage (disk); survives crashes
- Entry **committed** if known to be stored on majority of servers
(and leader has completed one entry in its current term)
 - Durable, will eventually be executed by state machines

Normal Operation

- Client sends command to leader
- Leader appends command to its log
- Leader sends AppendEntries RPCs to followers
- Once new entry committed: (majority? 7)
 - Leader passes command to local state machine, returns result to client
 - Leader notifies followers of committed entries in subsequent AppendEntries RPCs
 - Followers pass committed commands to their state machines
- Crashed/slow followers?
 - Leader retries RPCs until they succeed
- Performance is optimal in common case
 - One RPC to any majority of servers

- no extra prepare (Paxos)
- no extra heartbeats

Log Consistency

High level of coherency between logs:

- If log entries on different servers have same index and term:

- They store the same command/op

- The logs are identical

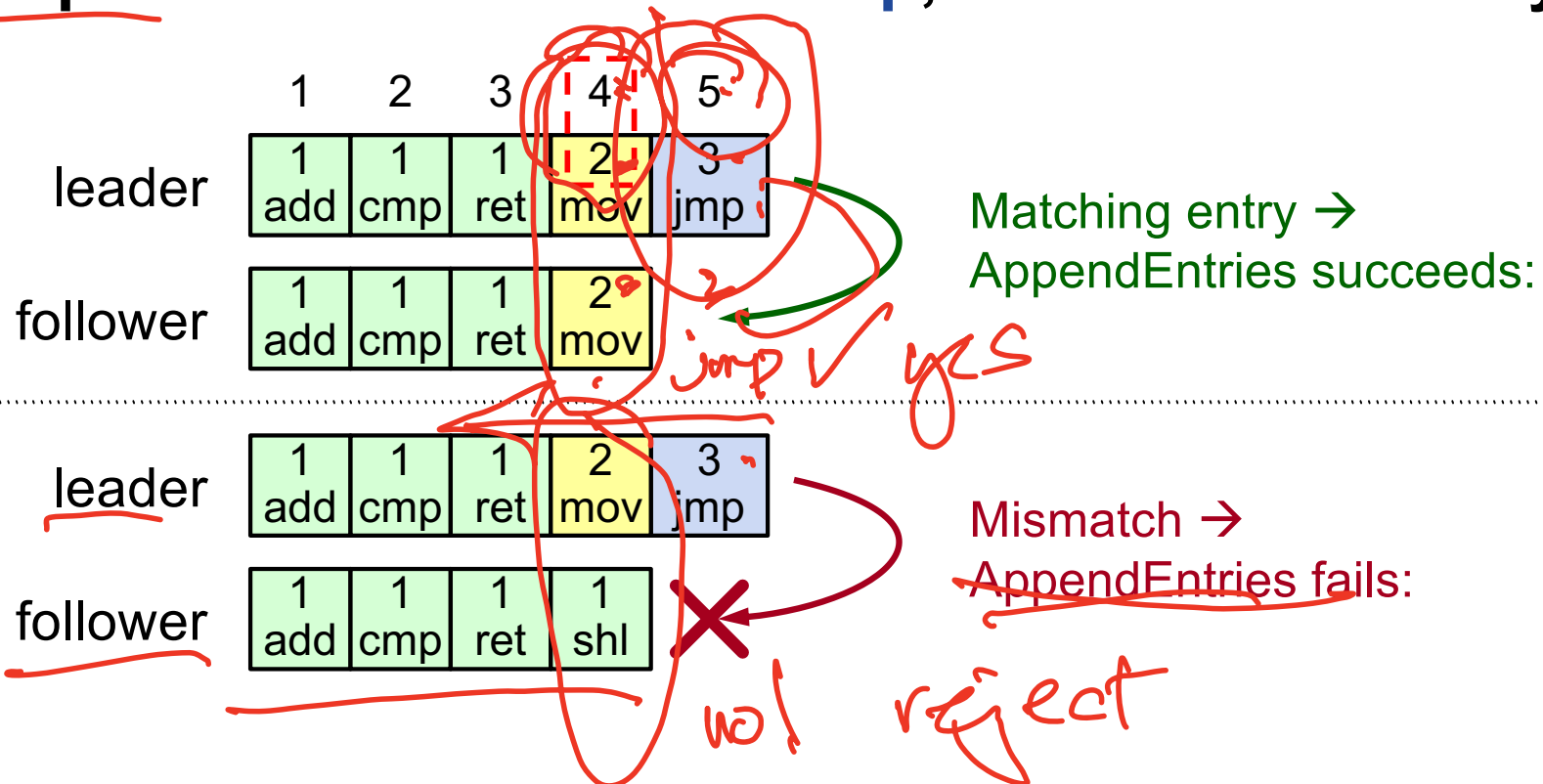
1	2	3	4	5	6
1 add	1 cmp	1 ret	2 mov	3 jmp	3 div
1 add	1 cmp	1 ret	2 mov	3 jmp	4 sub

whichever is leader, cleans up
→ make logs identical

- If a given entry is committed, all preceding entries are also committed

AppendEntries Consistency Check

- Each AppendEntries RPC contains *info about new cmd + ?*
→ index + term of preceding entry
- Follower must contain matching entry; otherwise it rejects request
- Implements an **induction step**, ensures coherency



Check: Log Properties?

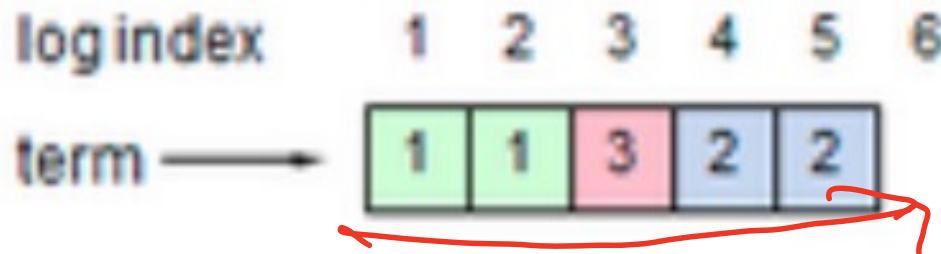
- Are the following log configurations possible?

a) No

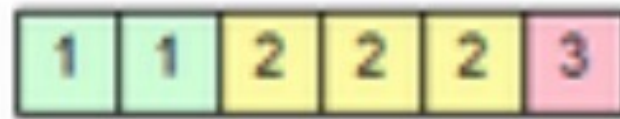
b) Yes

c) Yes

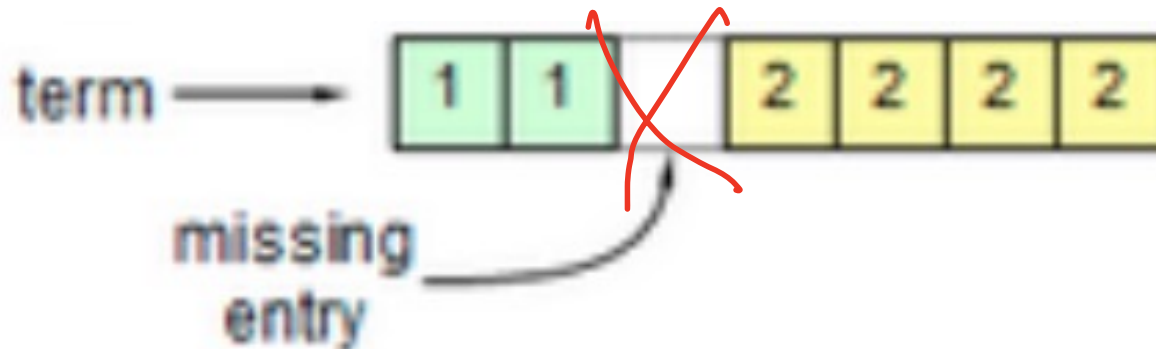
d) No



- or any
node
- independent



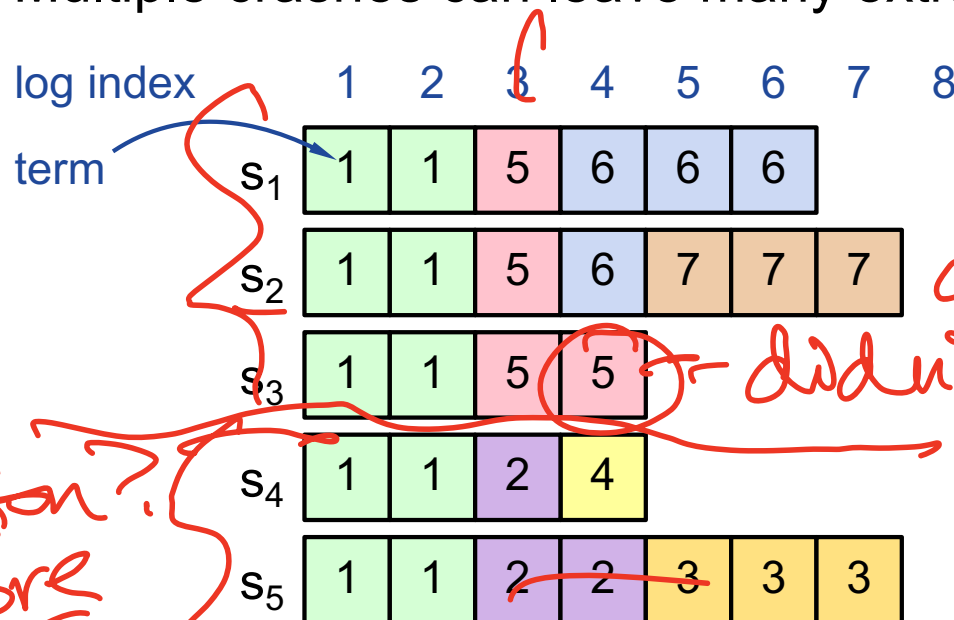
no leader elected? not
same
system



Leader Changes

- At beginning of new leader's term:

- Old leader may have left entries partially replicated
- No special steps by new leader: just start normal operation
- Leader's log is source of truth
- Leader will eventually make follower's logs identical to leader's
- Multiple crashes can leave many extraneous log entries:



partition before term 5

didn't commit in 7
didn't commit - no response to client

Safety Requirement

Once a log entry has been applied to a state machine, no other state machine must apply a different value for that log entry

- Raft safety property:

- If a leader has decided that a log entry is committed, that entry will be present in the logs of all future leaders

- must This guarantees the safety requirement

- Leaders never overwrite entries in their logs
- Only entries in leader's log can be committed
- Entries must be committed before applying to state machine

Committed → Present in future leaders' logs

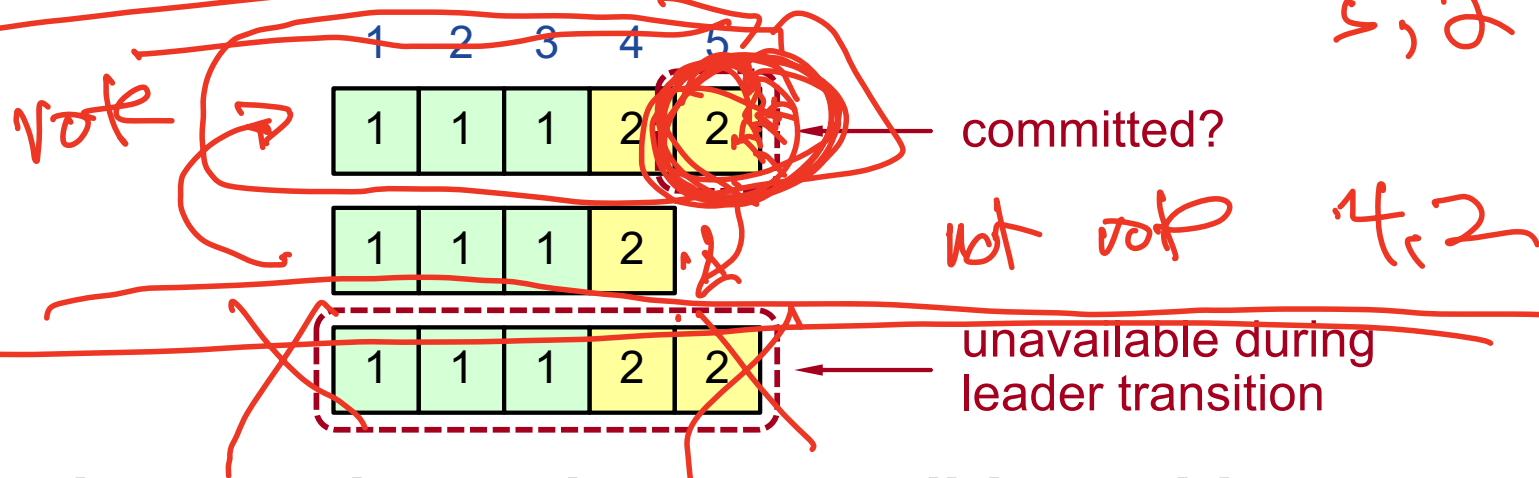
Restrictions on
commitment

Restrictions on
leader election

more complex
than majority of
nodes

Picking the Best Leader

- Can't tell which entries are committed!



- During elections, choose candidate with log most likely to contain all committed entries

- Candidates include log info in RequestVote RPCs (index & term of last log entry)
- Voting server V denies vote if its log is "more complete":
 $(\text{lastTerm}_V > \text{lastTerm}_C) \parallel$

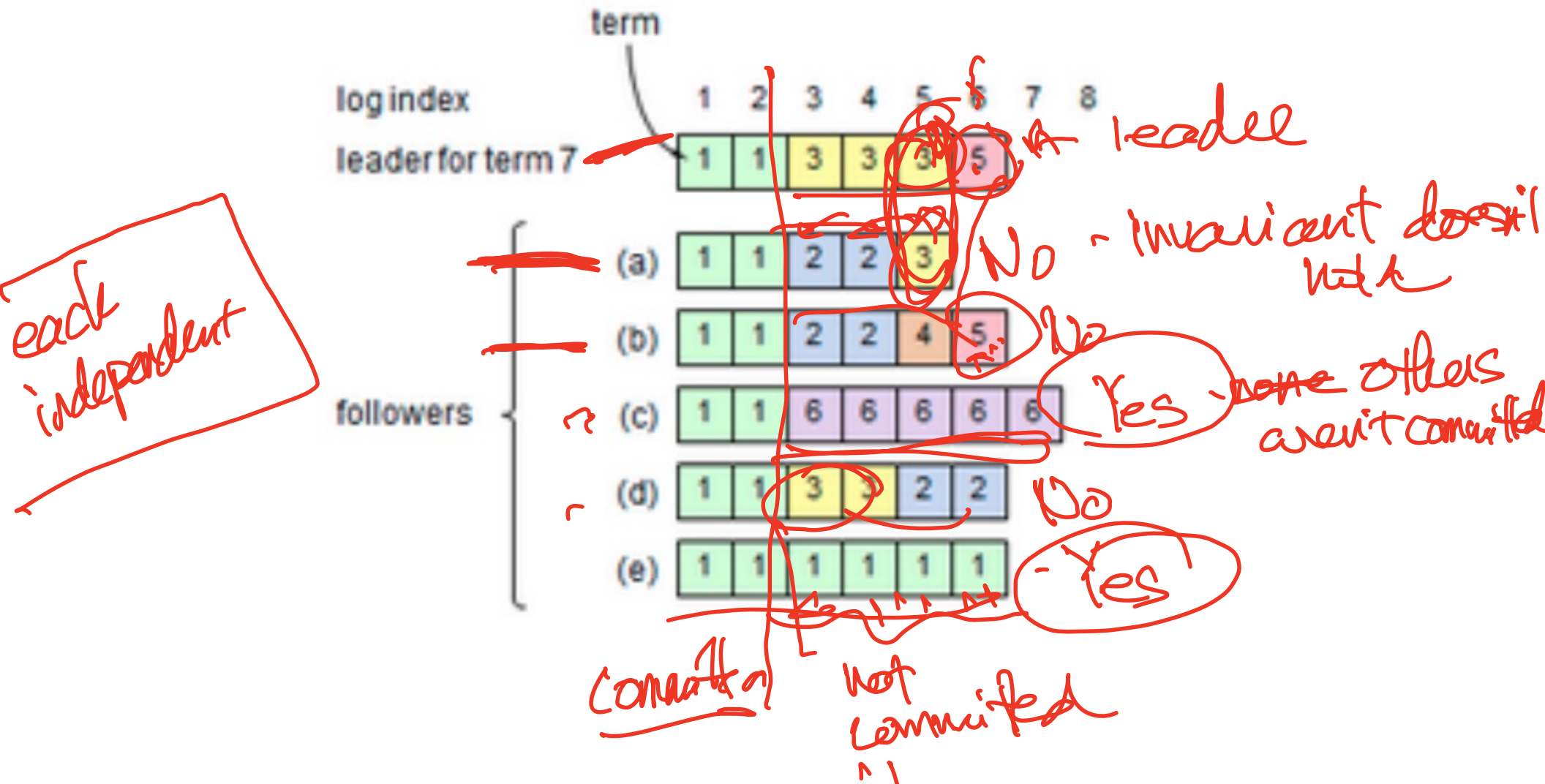
$\text{lastTerm}_V = \text{lastTerm}_C \ \&\& \ \text{lastIndex}_V >$

- Leader will have "most complete" log among electing majority

last
index

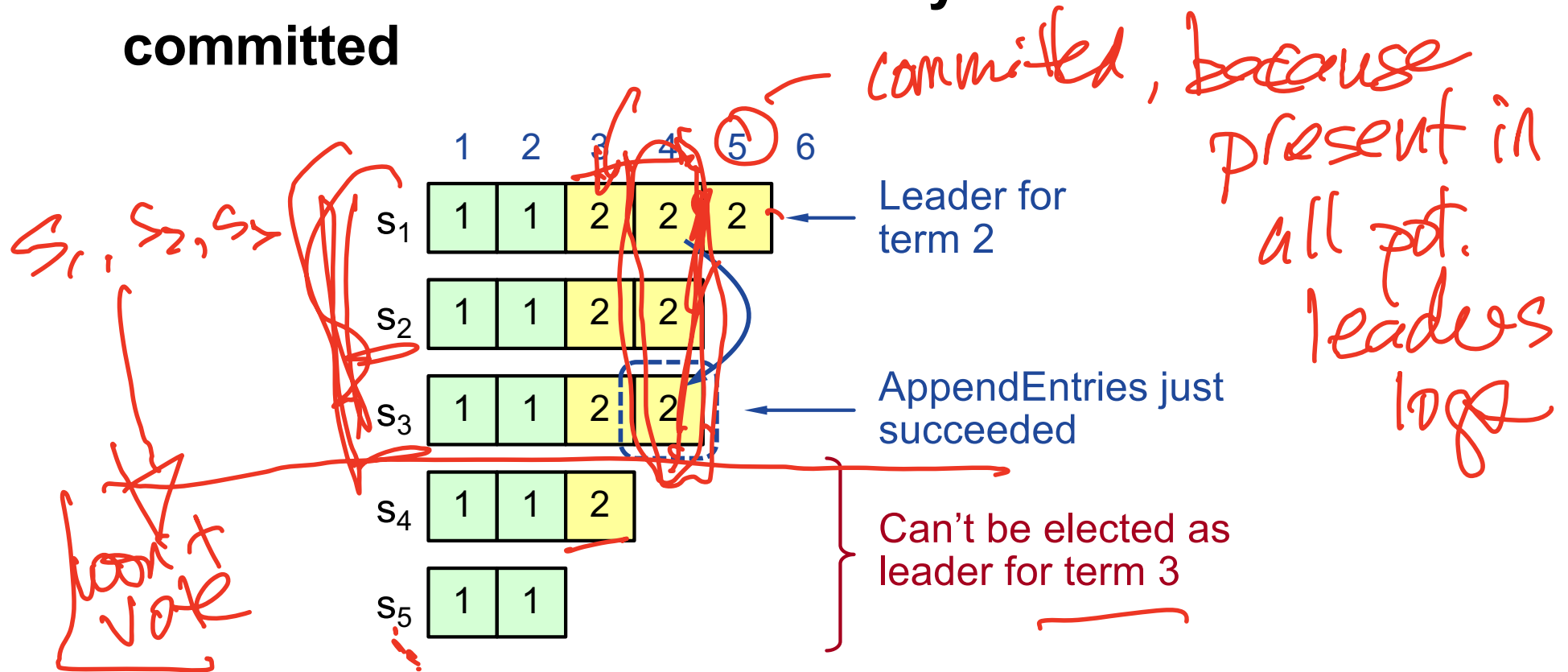
Check: Possible Followers?

For each of the followers, could the given log configuration occur?



Committing Entry from Current Term

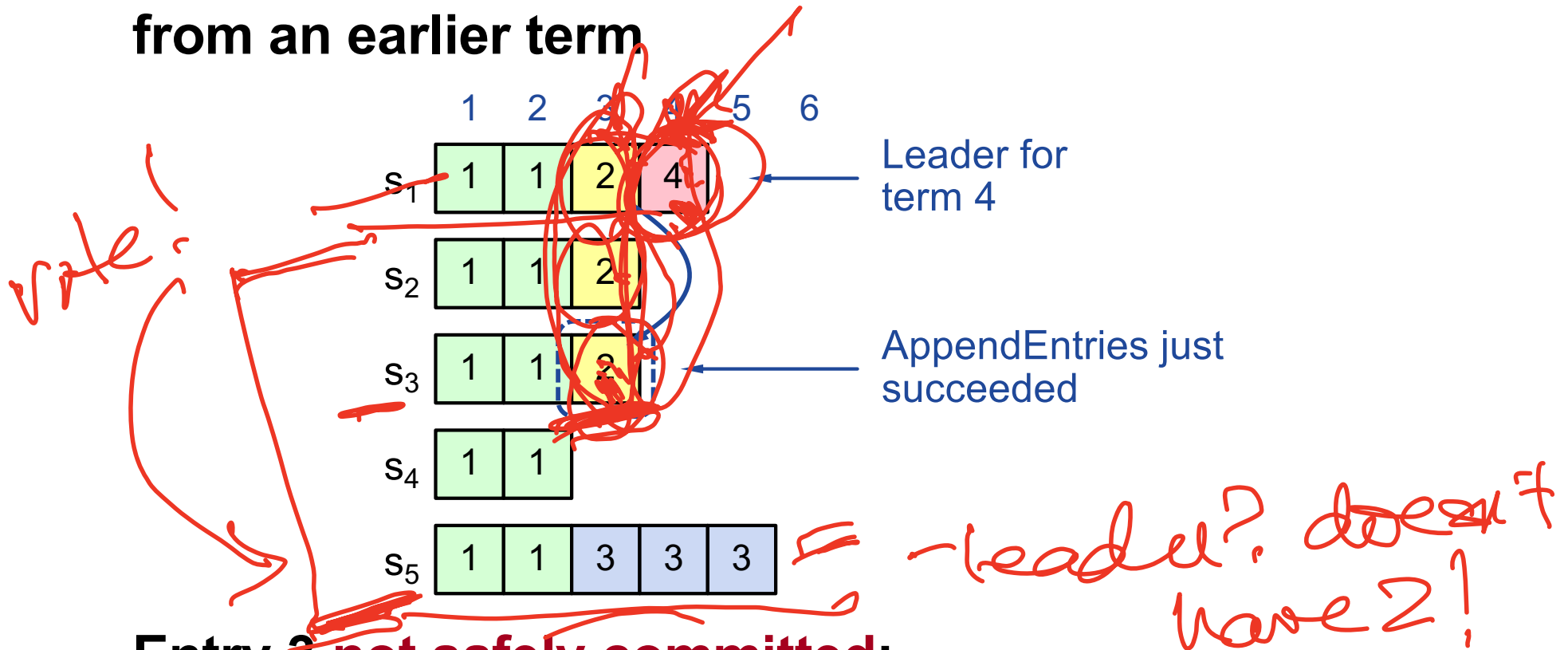
- **Case #1/2: Leader decides entry in current term is committed**



- **Safe: leader for term 3 must contain entry 4**

Committing Entry from Earlier Term

- **Case #2/2: Leader is trying to finish committing entry from an earlier term**



- **Entry 3 not safely committed:**

- s₅ can be elected as leader for term 5
- If elected, it will overwrite entry 3 on s₁, s₂, and s₃!

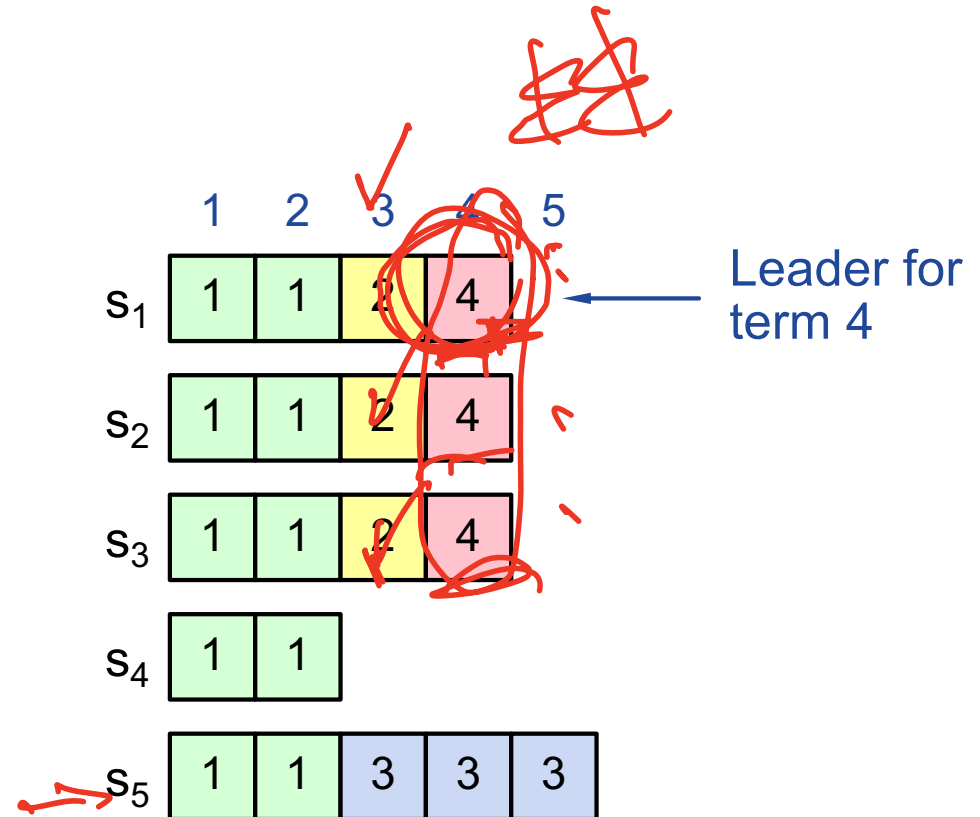
New Commitment Rules

- **For a leader to decide an entry is committed:**

- Must be stored on a majority of servers
- At least one new entry from leader's term must also be stored on majority of servers

- **Once entry 4 committed:**

- s₅ cannot be elected leader for term 5
- Entries 3 and 4 both safe

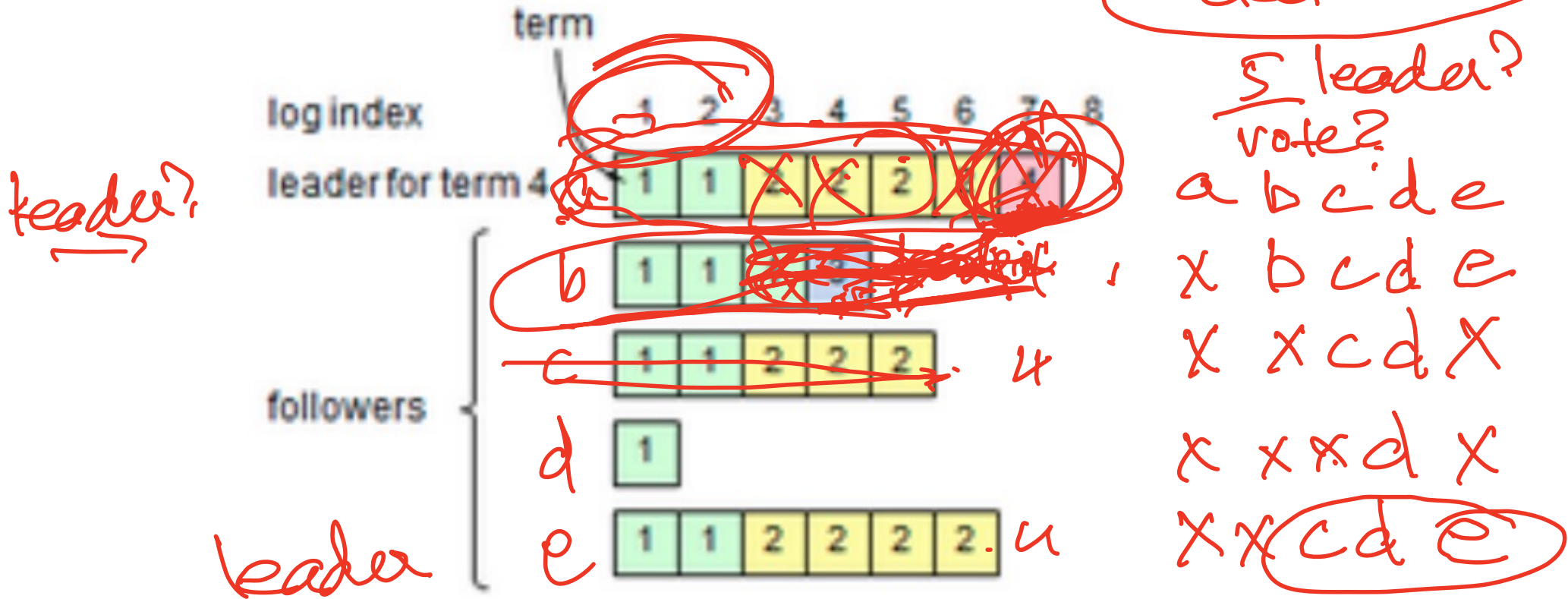


Combo of election and commitment rules makes Raft safe

nothing in term 2 is committed

Check: Committed Entries?

Which entries are committed and can be applied?



Hint: Where is there a majority? Who could be elected leader?

must show must be in
logs of any future leaders

Log Inconsistencies

Leader changes can result in log inconsistencies:

log index
leader for
term 8

log index	1	2	3	4	5	6	7	8	9	10	11	12
leader for term 8	1	1	1	4	4	5	5	6	6	6		

(a)

1	1	1	4	4	5	5	6	6		
---	---	---	---	---	---	---	---	---	--	--

(b)

1	1	1	4							
---	---	---	---	--	--	--	--	--	--	--

(c)

1	1	1	4	4	5	5	6	6	6	6
---	---	---	---	---	---	---	---	---	---	---

(d)

1	1	1	4	4	5	5	6	6	6	7	7
---	---	---	---	---	---	---	---	---	---	---	---

(e)

1	1	1	4	4	4	4				
---	---	---	---	---	---	---	--	--	--	--

(f)

1	1	1	2	2	2	3	3	3	3	3
---	---	---	---	---	---	---	---	---	---	---

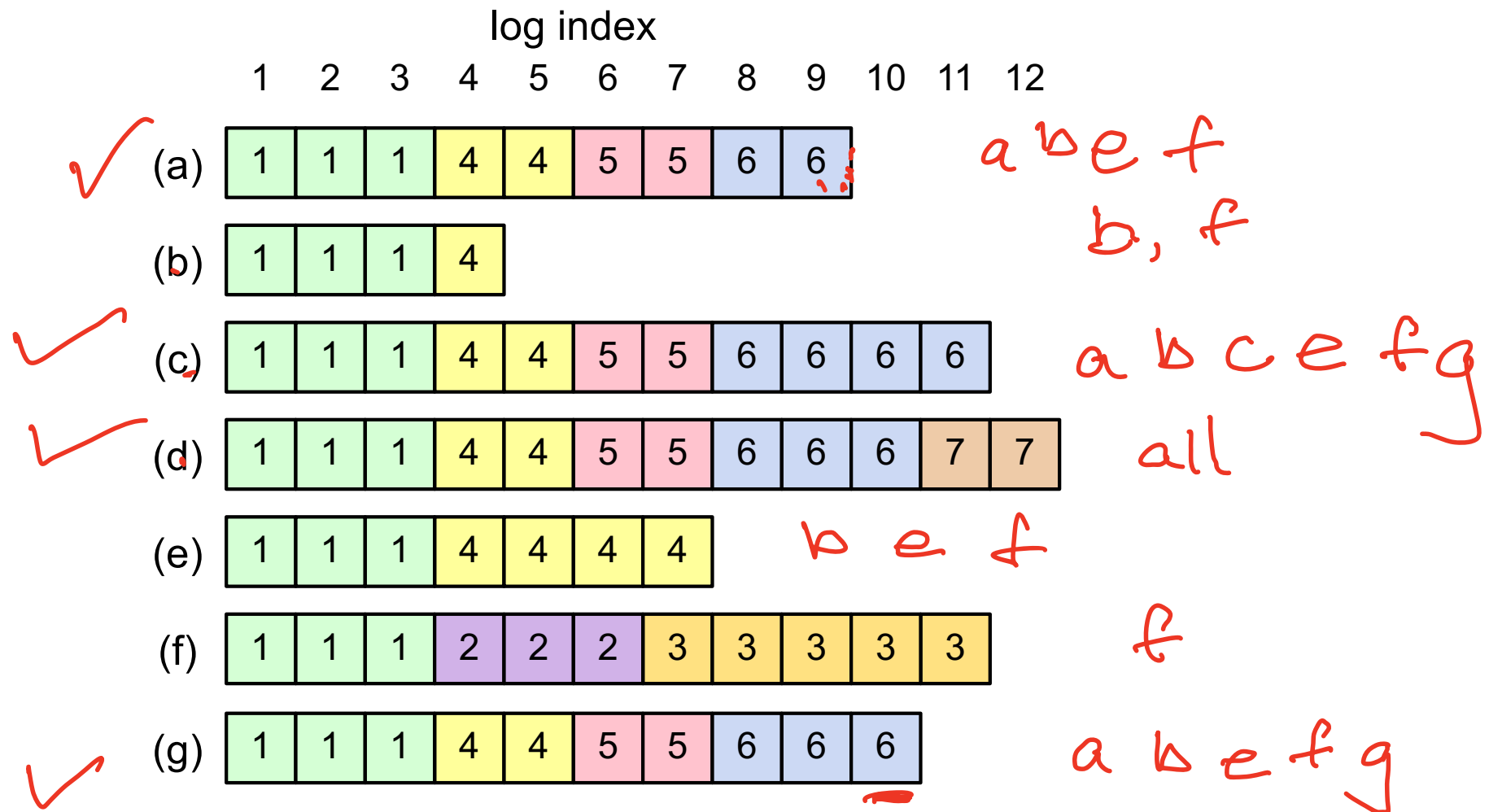
Missing
Entries

Extraneous
Entries

possible
followers

won't vote for

Who Could be Leader for Term 8?



Continue on Wed

Repairing Follower Logs

- New leader must make follower logs consistent with its own

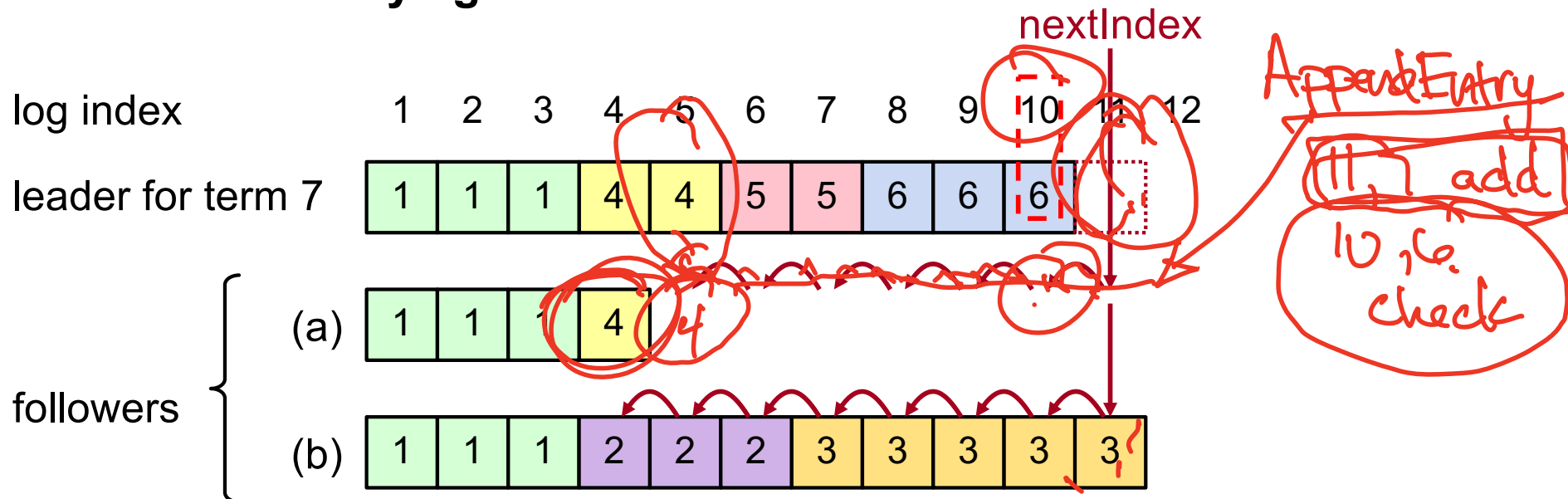
- delete
- fill in

2 mechanisms

- Leader keeps nextIndex for each follower:

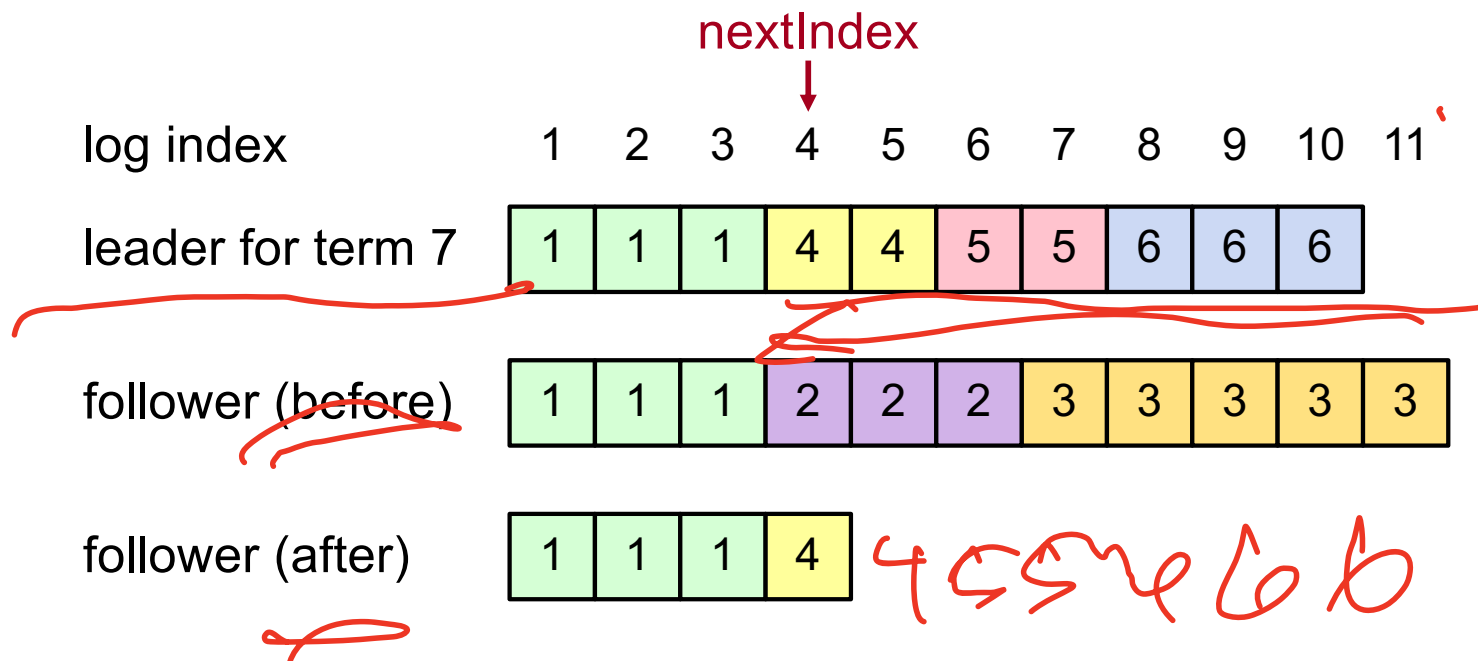
- Index of next log entry to send to that follower
- Initialized to $(1 + \text{leader's last index})$

- When AppendEntries consistency check fails, decrement nextIndex and try again:



Repairing Logs, cont'd

- When follower overwrites inconsistent entry, it deletes all subsequent entries:



Neutralizing Old Leaders

- **Deposed leader may not be dead:**
 - Temporarily disconnected from network
 - Other servers elect a new leader
 - Old leader becomes reconnected, attempts to commit log entries
- **Terms used to detect stale leaders (and candidates)**
 - Every RPC contains term of sender
 - If sender's term is older:
RPC is rejected sender reverts to follower and updates its term
- **Election updates terms of majority of servers**
 - Deposed server cannot commit new log entries

Client Protocol

- **Send commands to leader**
 - If leader unknown, contact any server
 - If contacted server not leader, it will redirect to leader
- **Leader does not respond until command has been logged, committed, and executed by leader's state machine**
- **If request times out (e.g., leader crash):**
 - Client reissues command to some other server
 - Eventually redirected to new leader
 - Retry request with new leader

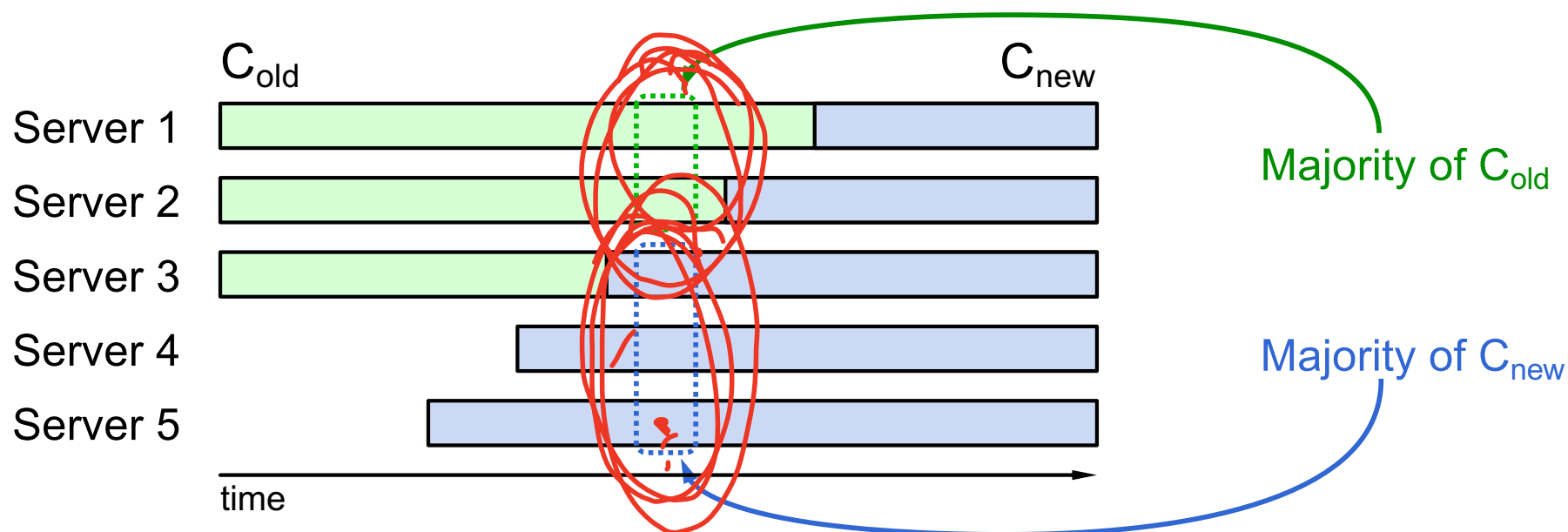
Client Protocol, cont'd

- **What if leader crashes after executing command, but before responding?**
 - Must not execute command twice
- **Solution: client embeds a unique id in each command**
 - Server includes id in log entry
 - Before accepting command, leader checks its log for entry with that id
 - If id found in log, ignore new command, return response from old command
- **Result: *exactly-once semantics* as long as client doesn't crash**

Configuration Changes

- Consensus mechanism must support changes in the configuration:
 - Replace failed machine; Change degree of replication

Cannot switch directly from one configuration to another: **conflicting majorities** could arise

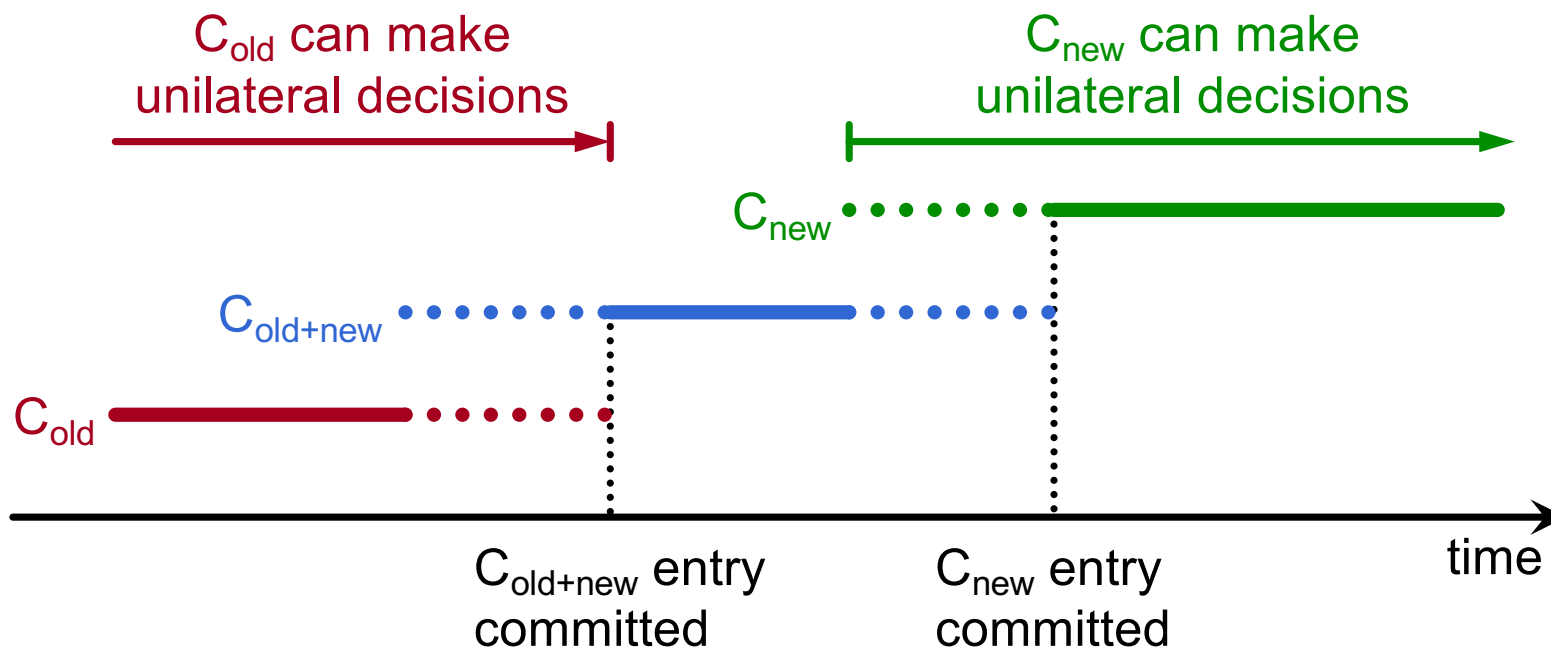


must not allow this! use joint consensus

Joint Consensus

- Raft uses a 2-phase approach:

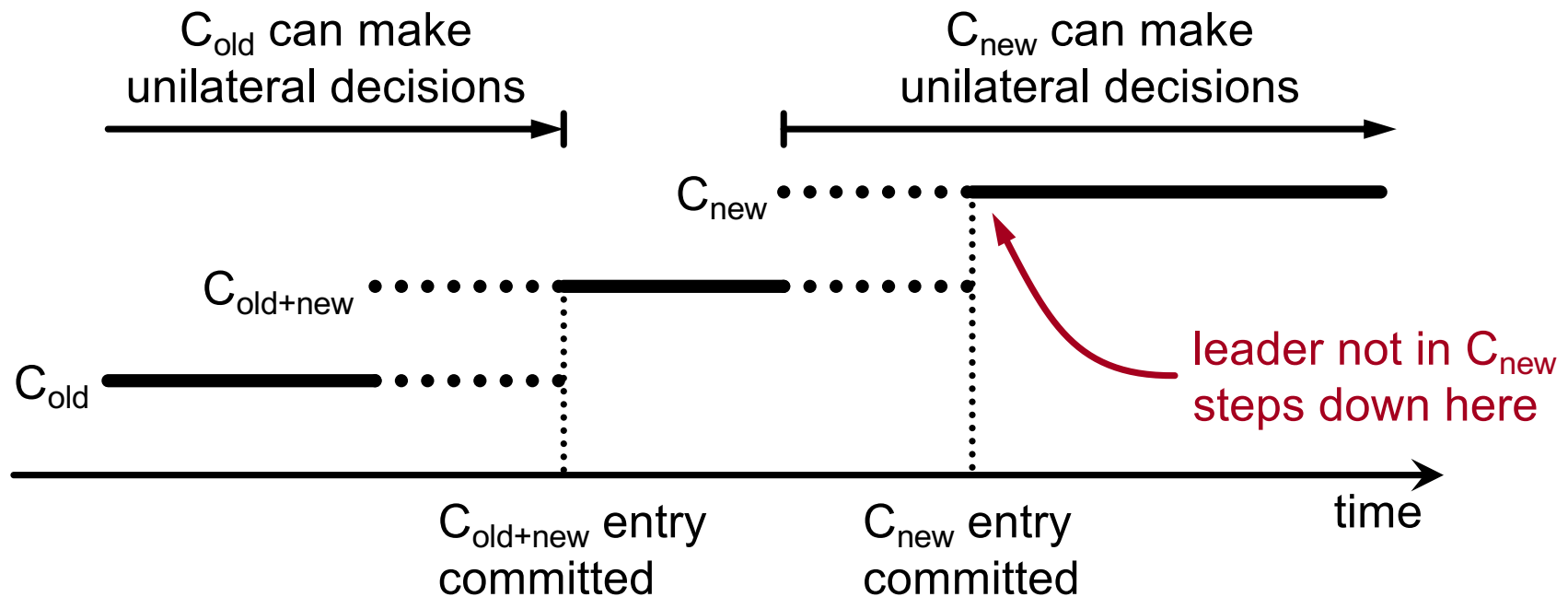
- Intermediate phase uses **joint consensus** (need majority of both old and new configurations for elections, commitment)
- Configuration change is just a log entry; applied immediately on receipt (committed or not)
- Once joint consensus is committed, begin replicating log entry for final configuration



Joint Consensus, cont'd

- **Additional details:**

- Any server from either configuration can serve as leader
- If current leader is not in C_{new} , must step down once C_{new} is committed.



- Might take awhile for new nodes to catch up w/ logs (act as non-voting)

Raft Summary

1. **Leader election**
2. **Normal operation**
3. **Safety and consistency**
4. **Neutralize old leaders**
5. **Client protocol**
6. **Configuration changes**

Other Issues?

- **Log compaction: Take periodic snapshots**
- **Is it really more understandable?**
- **Raft used for consensus in many real systems**
- **How can reads be handled?**

How to handle reads?

Goal: Strong consistency (and good performance)

- Appears as single copy
- Every op appears to have occurred at some instance in time between invocation and response

Approaches

1. Add Reads to log, achieve consensus, apply to RSM

- Strong consistency, but too much overhead

2. Other extreme: Send Reads to followers

- Not strong consistency; could read stale versions of data

3. Send Reads to leader

- How to ensure return only committed data?
 - Leader knows committed data; Might force leader to do extra work: ensure writes committed; no-op at start of term
- How to ensure still leader? Leader lease or heartbeat *to majority*